

Informe Exhaustivo de Ingeniería Arquitectónica y Plan Maestro de Refactorización Sistémica: Proyecto Resp (Transpilador Esperanto-Rust)

Introducción al Diseño de Compiladores Localizados y el Ecosistema DeepWiki

La hegemonía del idioma inglés en la sintaxis, las palabras clave y la infraestructura general de los lenguajes de programación ha creado históricamente una barrera cognitiva y educativa sustancial para los desarrolladores de software no angloparlantes en todo el mundo. Aunque han existido esfuerzos pioneros y experimentales para desarrollar lenguajes de programación en otros idiomas, tales como Rouille (una traducción directa de Rust al francés), Wenyan (basado íntegramente en la estructura del chino clásico) o Linotte, la inmensa mayoría de estas iniciativas sacrifican el rendimiento industrial, la seguridad de la memoria y la compatibilidad de ecosistemas a cambio de la accesibilidad puramente educativa. En contraposición a esta tendencia, el proyecto Resp emerge como una propuesta de ingeniería de compiladores de vanguardia. El objetivo fundamental de Resp es construir un lenguaje de programación moderno que ofrezca las garantías irrestrictas de seguridad de memoria del compilador oficial de Rust, incorpore la flexibilidad ergonómica y el paradigma orientado a objetos heredado de C++, y cuya base léxica y morfológica esté construida estrictamente utilizando el idioma internacional Esperanto.

El ecosistema actual del desarrollo de software no existe en el vacío, sino que está profundamente interconectado con plataformas de análisis impulsadas por inteligencia artificial. Una de las infraestructuras más relevantes en este contexto es DeepWiki, un servicio innovador lanzado por la organización Cognition AI el veintisiete de abril de dos mil veinticinco, construido sobre la plataforma AI Ascendancy. DeepWiki ha sido diseñado para abordar el problema crónico de la documentación incompleta o desactualizada en proyectos de código abierto, funcionando esencialmente como la enciclopedia de código definitiva o la "Wikipedia del código". Mediante la indexación de más de treinta mil repositorios públicos de GitHub y el procesamiento algorítmico de más de cuatro mil millones de líneas de código fuente, DeepWiki proporciona a los desarrolladores una experiencia de comprensión de código sin precedentes. La mecánica operativa de esta plataforma es notablemente elegante: los usuarios simplemente modifican la URL del repositorio original, sustituyendo el dominio de GitHub por deepwiki.com, para acceder instantáneamente a una interfaz interactiva y dinámica. En esta interfaz, en lugar de navegar manualmente por jerarquías de archivos y depender de búsquedas globales limitadas, los ingenieros pueden plantear preguntas en lenguaje natural y recibir respuestas precisas que están estrictamente fundamentadas en el código fuente real del proyecto. La premisa central que hace que DeepWiki sea revolucionario es que asume que el código fuente es la fuente de verdad absoluta e inmutable. Los Modelos de Lenguaje Grande (LLMs) subyacentes son cada vez más capaces de comprender bases de código en profundidad, lo

que significa que dirigirlos directamente hacia la sintaxis real produce resultados mucho más precisos que depender de documentación estática y derivada que a menudo envejece mientras el código evoluciona. Además, la plataforma expone un servidor bajo el Protocolo de Contexto de Modelos (Model Context Protocol, o MCP), el cual permite a agentes de inteligencia artificial y asistentes de codificación autónomos conectarse programáticamente a estas herramientas de interrogación de repositorios. El servidor MCP de DeepWiki proporciona acceso gratuito y sin autenticación a herramientas como la formulación de preguntas complejas, la lectura de contenidos de la wiki y la evaluación de la estructura del repositorio.

Sin embargo, esta dependencia absoluta en la veracidad del código fuente introduce un riesgo sistémico catastrófico para proyectos que se encuentran en estados de desarrollo inconsistentes. Como se documenta en incidentes recientes relacionados con repositorios de gran escala como LibreOffice, la generación de documentación impulsada por inteligencia artificial puede producir resultados engañosos si el código fuente contiene artefactos confusos o características implementadas a medias. En el caso de LibreOffice, la presencia de archivos residuales del sistema de construcción Buck provocó que la inteligencia artificial concluyera erróneamente que Buck era el sistema de construcción principal, propagando información falsa a usuarios novatos y generando la necesidad de solicitudes de eliminación (takedown requests) y correcciones manuales mediante comentarios en el código para guiar al modelo.

Al evaluar la madurez actual del proyecto Resp a través de la lente de las capacidades analíticas de DeepWiki, emerge una disonancia arquitectónica crítica. El proyecto Resp cuenta con un documento de diseño de investigación extenso y visionario que promete características avanzadas como herencia por composición, manejo de excepciones sin pánico mediante patrones de coincidencia, y sobrecarga dinámica de funciones. No obstante, el repositorio real del proyecto, estructurado en múltiples módulos de Rust (crates), presenta un motor de transpilación rudimentario, de un solo paso, que ignora deliberadamente la mayor parte de esta semántica avanzada. Si DeepWiki indexa el repositorio actual en su estado prototípico, el servidor MCP y los modelos de lenguaje analizarán el archivo de código transpilador y concluirán, correctamente, que características como la palabra clave `etendu` (extender) están inactivas y catalogadas como código muerto, contradiciendo directamente el manifiesto del lenguaje y creando una base de conocimiento fracturada y engañosa. Por lo tanto, es de máxima urgencia técnica que un agente de codificación autónomo, como OpenCode, intervenga en el repositorio para alinear la implementación física del AST y el transpilador con las especificaciones teóricas inmutables del lenguaje, garantizando así la madurez del proyecto y la integridad de su representación en los ecosistemas globales de inteligencia artificial.

Fundamentos Morfológicos y Evaluación del Analizador Léxico

El Esperanto, creado a finales del siglo diecinueve, es un idioma planificado que exhibe una regularidad morfológica y sintáctica absoluta, carente por completo de las excepciones irregulares que plagan a los lenguajes naturales. Esta consistencia matemática lo convierte en un candidato excepcional como base para un analizador léxico (lexer) y un analizador sintáctico (parser) en la ingeniería de compiladores. Las reglas lingüísticas dictaminan terminaciones fijas invariables para las categorías gramaticales: todos los sustantivos terminan en la vocal 'o', los adjetivos en 'a', los adverbios derivados en 'e' y los verbos en infinitivo en 'i'. Durante la fase crítica de tokenización del código fuente, esta predictibilidad estructural permite al analizador clasificar semánticamente los identificadores con una eficiencia algorítmica superior,

reduciendo drásticamente la ambigüedad computacional y facilitando la construcción de Árboles de Sintaxis Abstracta (AST) de muy alta fidelidad.

Para implementar este diseño léxico, el proyecto Resp utiliza la biblioteca pest, un potente generador de analizadores sintácticos en Rust basado en la Gramática de Expresiones de Análisis (Parsing Expression Grammar o PEG). El vocabulario informático integrado en el analizador se deriva de diccionarios técnicos estandarizados como el Plena Ilustrita Vortaro de Esperanto y el Komputada Leksikono, garantizando que el lenguaje sea comprensible para los hablantes sin necesidad de introducir neologismos no estandarizados.

La siguiente tabla ilustra la traducción implementada de las palabras clave de control de flujo y declaración estructural, evaluando su correspondencia entre Rust y Esperanto, así como su estado actual de integración en la gramática PEG del repositorio.

Palabra Clave (Rust)	Traducción (Esperanto)	Función Semántica en el Diseño	Estado en el Parser PEG Actual (esperanto.pest)
fn	fk	Declaración de funciones. Mantiene la brevedad ergonómica.	Implementado correctamente en fk_keyword y funkcio_decl.
let	estu	Verbo volitivo para la declaración y el enlace (binding) de variables.	Implementado en estu_decl permitiendo asignaciones opcionales.
mut	ŝanĝ	Raíz del verbo cambiar, denota la mutabilidad intrínseca de una referencia o variable.	Funcional en estu_decl y en las firmas de referencias en el sistema de tipos.
struct	strk	Abreviación de estructura, define tipos de datos compuestos.	Presente en strk_decl junto con la palabra clave opcional etendu para herencia.
impl	realigu	Modo volitivo para materializar o implementar el comportamiento de un rasgo o estructura.	Integrado en realigu_decl para contener bloques de funciones asociadas.
match	kongruu	Bifurcación basada en la evaluación exhaustiva y coincidencia de patrones.	Configurado en kongruu_expr y estructurado correctamente con brazos de coincidencia.
try / catch	provu / kaptu	Emulación de excepciones sobre el sistema de resultados de Rust, aislando los pánicos.	Deficiencia Severa: Definidos como tokens individuales, pero ausentes en las reglas sintácticas estructurales.

De manera análoga, la adaptación de los tipos primitivos requiere una traducción que intercepte

las declaraciones escalares y vectoriales de Rust y las transforme de manera segura. El compilador de Resp implementa un mapeo exhaustivo de estos tipos.

Tipo de Dato (Rust)	Tipo de Dato (Esperanto)	Mapeo AST (resp-ast/src/lib.rs)	Generación de Código (resp-transpiler)
i8 a i128, isize	ent8 a ent128, entg	Parseado en tipo_ent y asignado a enumeraciones estáticas como Tipo::Ent32.	Emisión de tokens exacta mediante cuasi-comillas a representaciones nativas i32, isize, etc.
u8 a u128, usize	nat8 a nat128, natg	Parseado en tipo_nat utilizando la raíz matemática "natura" para números sin signo.	Transcripción directa a tokens u8, u64, garantizando la seguridad de memoria sin firmar.
f32, f64	glit32, glit64	Identificados en tipo_glit representando "glitkomo" (coma flotante en el estándar IEEE-754).	Transformación fidedigna a primitivas de punto flotante de hardware.
String	Teksto	Nodo Tipo::Teksto para cadenas dinámicas almacenadas en el montículo (heap allocation).	Inyección de código automática .to_string() si se asignan literales de cadena estáticos a este tipo.
&str	&ĉen	Referencias de porción de cadena inmutables, utilizando el símbolo ampersand seguido de ĉen.	Análisis sintáctico recursivo que soporta referencias anidadas mutables e inmutables.
Vec<T>	Vektoro<T>	Tipos genéricos encapsulados iterativamente para representar colecciones contiguas en memoria.	Expansión recursiva del genérico interior preservando la sintaxis del tipo de vector nativo.

A pesar de la exhaustividad teórica de este esquema de tipos y palabras clave, el análisis del archivo `esperanto.pest` revela vulnerabilidades y omisiones gramaticales que paralizan el progreso del transpilador. La regla general `declaration`, que actúa como el punto de entrada principal para el análisis descendente, agrupa las sentencias funcionales pero omite por completo cualquier referencia a bloques de manejo de errores, a pesar de que la documentación del diseño los considera un pilar de la ergonomía emulada de C++. Además, las reglas sintácticas para las expresiones matemáticas, tales como `cmp_expr`, `add_expr`, y `mul_expr`, aunque funcionales, introducen un sistema de precedencia de operadores frágil que debe ser respaldado por un Árbol de Sintaxis Abstracta (AST) matemáticamente estricto.

Diagnóstico Arquitectónico del Árbol de Sintaxis

Abstracta y el Transpilador

El proceso de transpilación en Resp no opera emitiendo representaciones intermedias de LLVM (LLVM-IR), sino transcribiendo el código nativo a flujos de tokens seguros de Rust (TokenStream) utilizando las formidables bibliotecas del ecosistema interno: syn, quote, y proc-macro2. Esta arquitectura se divide lógicamente en módulos estancos. Una vez que la cadena de texto cruda es tokenizada, toma el control el crate resp-parser para instanciar el Árbol de Sintaxis Abstracta definido en el crate resp-ast.

El AST no es una mera formalidad, sino la representación jerárquica y semántica de toda la lógica del programa. Se define mediante el enumerador Esprimo (Expresión) y el enumerador Deklaro (Declaración). Para garantizar que los errores generados por el compilador final de Rust puedan mapearse visualmente de regreso al código en Esperanto original, los diseñadores encapsularon todos los tipos hoja del AST en un contenedor genérico denominado Located<T>, el cual almacena el valor semántico junto con una tupla Span que registra la línea y la columna exactas del token inicial y final en el documento fuente.

Sin embargo, una inspección minuciosa del motor de generación de código, ubicado en crates/resp-transpiler/src/lib.rs, expone fallas sistémicas que invalidan la propuesta de valor del lenguaje. La transpilación se ejecuta mediante un recorrido recursivo del AST donde cada nodo invoca la macro quote! para emitir código fuente Rust. Este enfoque, aunque directo, ha resultado en una arquitectura de "pasada única" (single-pass compilation) que carece totalmente de un analizador semántico. En la teoría moderna de compiladores, la omisión de una fase de análisis semántico —que debería generar y poblar una Tabla de Símbolos global (Symbol Table) con las firmas de todas las funciones, tipos de retorno, y estructuras de datos antes de generar el código final— hace que sea computacionalmente imposible implementar la resolución de contextos complejos.

Esta deficiencia estructural engendra las tres disonancias críticas que separan al proyecto de un estado de madurez industrial y que deberán ser resueltas por el agente autónomo OpenCode:

Disonancia 1: La Ilusión de la Herencia Sintáctica por Composición

El requerimiento cardinal del diseño de Resp es dotar al lenguaje de la fluidez característica del paradigma orientado a objetos de C++, contrarrestando la estricta composición arquitectónica impuesta por Rust. Para lograr esto, se prometió que una estructura (declarada mediante la palabra clave strk) podría heredar los atributos de otra estructura utilizando la palabra clave de extensión etendo. Durante la transpilación, este concepto de herencia debería expandirse de manera invisible inyectando todos los campos de la estructura padre en el interior de la estructura hija y generando automáticamente bloques impl con delegaciones de métodos subyacentes, logrando una herencia simulada de costo cero en tiempo de ejecución.

Al revisar la transpilación de Deklaro::Strk en el código actual, se observa la siguiente implementación de abandono técnico :

```
if let Some(_ext) = etendo {  
    // Inheritance via composition: will be expanded  
}
```

El transpilador, al carecer de una Tabla de Símbolos, no tiene manera de consultar cuáles son los campos de la estructura padre nombrada en la variable etendo. Por lo tanto, el transpilador

simplemente inserta un comentario vacío en su propio código fuente, descarta silenciosamente la instrucción del programador e ignora la herencia, rompiendo los principios fundamentales del diseño del lenguaje y dejando una estructura inoperante.

Disonancia 2: La Falsa Sobrecarga de Funciones y Dispatch Dinámico

Los lenguajes como C++ permiten la definición de múltiples funciones que comparten el mismo identificador léxico siempre que la aridad o el tipo de sus argumentos difieran, un concepto conocido como sobrecarga de funciones o polimorfismo ad hoc. Rust, por decisiones conscientes de diseño orientadas a la claridad extrema, rechaza la sobrecarga de funciones nativa, obligando al uso de constructores complejos o macros. El informe de diseño dictamina que Rust debe ofrecer soporte nativo para la sobrecarga calculando las firmas de las funciones y aplicando una técnica de manipulación de identificadores (name mangling), o emitiendo automáticamente una macro declarativa de Rust que redirija la llamada en el backend de forma transparente.

En el AST y el transpilador actuales, la variante `Deklaro::Funkcio` transcribe el identificador de la función exactamente como se recibe desde el parser de Esperanto, salvo por una regla codificada estáticamente que traduce el identificador `ĉefa` (principal) a `main` para cumplir con el punto de entrada requerido por el binario de Rust. Si un programador declara `fk procesi(n: ent32)` y luego `fk procesi(n: glit64)`, el transpilador generará ciegamente dos funciones llamadas `procesi` en el mismo ámbito léxico de Rust. El compilador `rustc` fallará inevitablemente con un error de identificador duplicado, demostrando que la característica de sobrecarga de funciones es, en la versión actual, una completa fabricación.

Disonancia 3: La Evaporación Silenciosa del Manejo de Excepciones

Mientras que Rust fuerza la evaluación exhaustiva de los posibles caminos de falla a través de su enumeración `Result<T, E>` y el operador de retorno anticipado, C++ y muchos lenguajes de alto nivel utilizan un flujo de control de bloque basado en sentencias `try` y `catch`. El diseño establece la creación de los constructos análogos `provu` y `kaptu` en Esperanto. Teóricamente, el transpilador debería convertir estos bloques en un elegante manejo de correspondencia de patrones subyacente que garantice la captura de los errores y evite el colapso (`panic`) de la aplicación.

El AST posee la variante `Esprimo::Provu(Vec<Deklaro>, Option<Vec<Deklaro>>)`, diseñada de forma fundamentalmente incorrecta, ya que asume que el contenido del bloque `try` es meramente una lista de declaraciones sueltas sin valor de retorno, imposibilitando la asignación de la expresión evaluada. Peor aún, el código de generación asociado en el transpilador revela la negligencia más alarmante del proyecto:

```
Esprimo::Provu(try_block, catch_block) => {
    let t: Vec<_> = try_block.iter().map(|d|
transpile_deklaro(d)).collect();
    match catch_block {
        Some(_c) => {
            quote! { { #(t)* } }
        }
        None => {
```

```

    quote! { { #(#t)* } }
  }
}

```

Independientemente de si existe o no un bloque `catch` (`Some(_c)`), la variable que lo contiene jamás es evaluada o transpilada. El transpilador escupe únicamente las sentencias del bloque `try`, desechando por completo cualquier lógica de recuperación de fallas redactada por el usuario, sin emitir siquiera una advertencia en la consola de compilación.

Disonancia 4: Ruptura de la Trazabilidad Espacial y Spans Corruptos

Uno de los pilares tecnológicos del transpilador radica en la preservación de los metadatos posicionales (`Spans`) originales. Cuando el compilador final de Rust detecta una violación de seguridad de memoria, el transpilador debe interceptar ese mensaje y proyectar la advertencia directamente sobre la línea exacta del código fuente original en Esperanto (.resp). A lo largo del archivo `resp-transpiler/src/lib.rs`, el desarrollador original instanció todos y cada uno de los identificadores sintácticos de Rust utilizando el método `proc_macro2::Span::call_site()`. Este método asigna un indicador espacial genérico que carece de resolución de líneas, vinculando todos los errores de la compilación generada exclusivamente al archivo intermedio auto-generado (.rs), destruyendo la abstracción y forzando al usuario a depurar código que él mismo nunca escribió. Además, el envoltorio de la Interfaz de Línea de Comandos (CLI) alojado en `crates/resp-cli/src/main.rs` ejecuta `rustc` en un proceso hijo que devuelve las cadenas de error estándar en bruto, sin ningún intento de parsear, filtrar o redirigir dichos errores mediante un mapa de fuentes estructurado.

La Perspectiva de Maduración: Integración Autónoma a través de Claude Code y Hooks

Llevar a la realidad la totalidad de la arquitectura de este compilador y rectificar simultáneamente las cuatro disonancias sistémicas expuestas representa un esfuerzo volumétrico inmenso, particularmente porque la reestructuración requerida debe ser orquestada por asistentes de terminal como OpenCode o Claude Code. Las operaciones a nivel de compiladores, como el rediseño de las gramáticas formales PEG y el recorrido transversal profundo del AST, exigen un contexto ininterrumpido a lo largo de docenas de iteraciones complejas.

Los agentes conversacionales basados en línea de comandos están sujetos a un fenómeno conocido como la "compactación automática del contexto" (auto-compaction). Cuando el intercambio masivo de lectura y escritura de múltiples archivos oxidados (archivos fuente de Rust) acerca la sesión computacional a su capacidad máxima de retención de tokens, el agente desencadena una purga de la memoria detallada para evitar la terminación abrupta del proceso y reemplaza el historial técnico con resúmenes abstractos. En el ámbito del diseño de lenguajes, la pérdida de directivas específicas sobre el mapeo léxico del Esperanto o la sutil jerarquía de dependencias del analizador `syn` inducirá inevitablemente a la generación de código alucinado que corromperá el frágil motor de macros.

Para blindar la ejecución del plan de refactorización contra las limitaciones estocásticas del modelo, la metodología de ejecución debe integrar tácticas avanzadas. El agente deberá

generar un archivo inmutable de reglas maestras en la raíz del proyecto antes de alterar la lógica del AST, garantizando una base de datos estática que funcione como faro de recuperación. Adicionalmente, el ecosistema exige la manipulación de la configuración profunda del agente (archivos JSON locales) para instalar "Hooks" o ganchos de interceptación. Un gancho de validación continua (PostToolUse) será acoplado a las operaciones de escritura; tras cada inyección de lógica de transpilación, el entorno forzará de manera transparente la invocación de cargo check y cargo fmt en el subproceso, enviando la salida de error estándar (stderr) directamente de vuelta a la ventana de razonamiento del agente. Esta red de seguridad cibernética es el único mecanismo confiable que previene la acumulación exponencial de deuda técnica y fallos tipográficos en las implementaciones de cuasi-comillas interpoladas de quote!.

El éxito de la migración del repositorio desde un estado de prototipo estructural fracturado hacia una madurez operativa completa dependerá de la adherencia paramétrica estricta y secuencial al plan de diseño expuesto en la siguiente sección.

Plan Maestro de Ejecución Secuencial y Resoluciones Arquitectónicas para Agentes Autónomos

Las instrucciones presentadas a continuación constituyen una especificación técnica de ejecución estricta, diseñada para que un agente autónomo como OpenCode evalúe el informe de investigación, decodifique la estructura de deficiencias subyacentes, y altere el código fuente del repositorio Resp paso a paso. Se prohíbe la omisión de cualquier fase. Cada segmento debe completarse, y los comandos de validación estructural deben confirmar la integridad sintáctica antes de progresar a la siguiente capa arquitectónica.

Fase Inicial: Establecimiento del Entorno y Reglas de Persistencia

Antes de manipular la sintaxis del analizador, la preservación del estado es crítica para garantizar que las modificaciones posteriores no fragmenten la comprensión arquitectónica del repositorio.

1. **Construcción de la Memoria Inmutable:** El agente deberá crear un archivo denominado CLAUDE.md en el directorio raíz. Este manifiesto declarativo contendrá un glosario absoluto de las conversiones léxicas (ejemplo, cartografiando el token `fk` hacia `fn`, y `estu` hacia `let`) junto con el mandato inquebrantable de utilizar la biblioteca `pest` para la resolución léxica y el macro `quote!` para la emisión sincrónica del código Rust. La existencia de este archivo asegurará el reenfoque automático del modelo de lenguaje tras cualquier evento de compactación de contexto.
2. **Activación de los Ganchos de Validación Continua (Hooks):** Manipulando las configuraciones del entorno o mediante scripts del sistema temporal, el agente configurará una directiva automatizada que ejecute obligatoriamente `cargo fmt` y `cargo check --workspace` cada vez que se detecte una escritura en archivos con terminación `.rs` y `.pest`.

Fase 1: Implementación del Analizador Semántico Multipaso (resp-sema)

El defecto más paralizante del proyecto, responsable de las disonancias de la herencia y la sobrecarga, es la ausencia de un recolector de metadatos previo a la generación de código. El agente debe transicionar el compilador de una pasada simple a una arquitectura de dos fases.

1. **Inyección del Nuevo Crate:** Modificar el archivo Cargo.toml raíz para incluir un nuevo miembro en el espacio de trabajo. Añadir "crates/resp-sema" bajo el arreglo members.
2. **Estructuración de la Tabla de Símbolos:** En la carpeta crates/resp-sema/src/lib.rs, instanciar la biblioteca principal que definirá la estructura SymbolTable. Esta tabla de símbolos contendrá mapas de dispersión dinámicos (HashMaps) que vinculen cadenas de texto identificadoras con firmas tipadas.
3. **Algoritmo de Recorrido Previo:** Programar una función pública iterativa analyze_program(prog: &mut Programa) -> SymbolTable. Esta función descenderá recursivamente por el vector de declaraciones del Árbol de Sintaxis Abstracta (AST) extrayendo el nombre de cada estructura (strk) y la lista exhaustiva de sus campos y tipos asociados, insertándolos en el registro global.
4. **Generación de Firmas Mutiladas (Name Mangling):** Dentro de la función de análisis semántico, implementar un detector de colisiones. Cuando se registren declaraciones múltiples de la variante Deklaro::Funkcio que compartan idéntico identificador léxico en nomo.value, el analizador mutará el AST en tiempo de ejecución, reescribiendo la propiedad nomo.value hacia una cadena concatenada estandarizada (ejemplo: nomoriginal_ent32_glit64), resolviendo permanentemente la incapacidad de sobrecarga del entorno Rust sin requerir bibliotecas adicionales.

Fase 2: Rectificación Profunda y Expansión de la Gramática Formal PEG

El archivo que fundamenta la lógica de análisis, crates/resp-parser/grammar/esperanto.pest, contiene omisiones que paralizan la funcionalidad del control de errores y valores por defecto. El agente debe reescribir estas reglas con extrema precisión.

1. **Integración de Cláusulas de Excepción:** Introducir la sentencia de prueba formalizando el uso de las palabras clave provu y kaptu.
 - Añadir la alternativa | provu_stmt dentro del bloque expansivo declaration.
 - Definir la regla sintáctica combinada: provu_stmt = { provu_keyword ~ bloko ~ (kaptu_keyword ~ bloko)? } asegurando que el bloque catch es condicional mediante el sufijo de interrogación opcional.
2. **Parámetros Modificables por Defecto:** Modificar la regla de análisis de argumentos para absorber sobrecargas emuladas. Alterar param = { identigilo ~ ":" ~ tipo } transformándola rigurosamente hacia param = { identigilo ~ ":" ~ tipo ~ ("=" ~ expr)? }.
3. **Purga de Residuos Fantasma:** Los tokens fantasma documentados en el Árbol Sintáctico Abstracto, como las primitivas imaginarias kreis y kreu, deben ser ignoradas de la sintaxis pest, y posteriormente eliminadas del árbol para evitar discrepancias irresolubles.

Fase 3: Reestructuración y Enriquecimiento del Árbol de Sintaxis Abstracta

La representación in-memory del lenguaje, definida en crates/resp-ast/src/lib.rs, debe ser sincronizada con la nueva gramática expandida.

1. **Soporte de Atributos Defecto en Parámetros:** Expandir la estructura subyacente de parámetros:

```
#
pub struct Parametro {
    pub nomo: Located<String>,
    pub tipo: Option<Tipo>,
    pub valor_defecto: Option<Esprimo>,
}
```
2. **Transformación del Nodo Provu:** Alterar radicalmente la definición morfológica de las excepciones. La definición previa defectuosa `Provu(Vec<Deklaro>, Option<Vec<Deklaro>>)` debe sustituirse por `Provu(Box<Esprimo>, Option<Vec<Deklaro>>)` garantizando que el bloque de intentos actúe computacionalmente como una expresión funcional capaz de retornar un `Result` empaquetado, en lugar de un mero vector secuencial y mudo de declaraciones procedimentales.
3. **Saneamiento Estructural:** Localizar el enumerador iterativo `Esprimo` y suprimir completamente las variantes `Kreis(Vec<Deklaro>)` y `Kreu(Vec<Deklaro>)`, saneando la disparidad entre los analizadores.

Fase 4: Adaptación del Motor de Conversión de Análisis Sintáctico

La función orquestadora `process_declaration` localizada dentro del archivo fuente `crates/resp-parser/src/lib.rs` exige refactorización para interpretar exitosamente las tuplas recién formadas por la biblioteca `Pest`.

1. **Resolución de Bloques de Excepciones:** Añadir un nuevo brazo de coincidencia de patrones (match arm) dedicado a la iteración sobre `Rule::provu_stmt`. Esta sección extraerá lógicamente los pares léxicos, comprobando la presencia del token opcional `kaptu_keyword`. Seguidamente, mapeará recursivamente las listas interiores invocando la subrutina `parse_block`, y retornará el nuevo envoltorio abstracto definido previamente, `Deklaro::EsprimoStmt(Esprimo::Provu(...))`.
2. **Asignación Paramétrica Expresiva:** En el bloque computacional que iterativamente ensambla la regla `Rule::param`, añadir una evaluación en bucle que detecte el operador matemático de asignación. Si se identifica, delegar el valor continuo a la función `parse_expr` y empaquetar el objeto resultante dentro del campo recién agregado `valor_defecto`.

Fase 5: Revolución del Motor de Cuasi-Comillas e Implementación de la Ergonomía C++

El núcleo tecnológico del proyecto, el archivo `crates/resp-transpiler/src/lib.rs`, concentra la inmensa mayoría de las deudas técnicas y falsas implementaciones detalladas en la investigación. La reestructuración de este bloque garantiza el paso de la teoría a la compilación efectiva.

1. **Simulación Compleja de Herencia Estructural mediante Composición Activa:** Al transicionar por la variante de estructuración de clases `Deklaro::Strk`, la existencia de un valor positivo en la variable opcional `etendo` debe desencadenar la inserción estructural

obligatoria.

- El agente debe eliminar el bloque de comentario inútil.
- Debe invocar la tabla de símbolos proporcionada por la capa de análisis semántico (resp-sema) para validar la existencia de la estructura identificada como el nodo padre.
- Durante la inyección macroscópica efectuada por la biblioteca quote!, el motor anexará físicamente el campo privado abstracto `__parent: #padre_ident` en el núcleo interno de la estructura generada.
- Inmediatamente a continuación, el transpilador debe escupir programáticamente y de forma automatizada las definiciones integrales para los rasgos dinámicos de desreferenciación intrínsecos de Rust, asegurando el polimorfismo morfológico deseado. El código inyectado deberá materializarse en tiempo de transpilación simulando exactamente las delegaciones implícitas `impl std::ops::Deref` y `impl std::ops::DerefMut` sobre la estructura heredera apuntando matemáticamente hacia el atributo interno subyacente `__parent`.

2. **Consolidación del Cierre y Retorno Seguro en Bloques de Excepción:** La disfunción donde la lógica del receptor de captura (kaptu) es silenciosamente obliterada debe cesar inmediatamente. El brazo generador para `Esprimo::Provu(try_block, catch_block)` será sobrescrito completamente.
 - El motor de transpilación utilizará el paradigma de Cierres Invocados Inmediatamente (Immediately Invoked Closures) característico de las aplicaciones maduras en Rust.
 - El macro envolverá la transpilación de las declaraciones internas del segmento de intento dentro de una expresión funcional sin argumentos que especifique como valor de retorno una variante empaquetada globalmente: `|| -> Result<_, Box<dyn std::error::Error>> { Ok(#try_expr) }`.
 - La invocación exterior de este cierre transitorio será sometida a una operación algorítmica constructiva de filtrado de patrones (match arm generation) que decida elegantemente: si se arroja la variante `Ok(val)`, retornará imperativamente el valor puro; en la eventualidad perniciosa de un colapso materializado en `Err(_)`, la macro ejecutará sin fallos el ciclo de sentencias almacenado preventivamente dentro del bloque iterativo extraído desde el token condicional `#catch_block`.

Fase 6: Solución de Trazabilidad Cartográfica Espacial y Retroalimentación CLI

Para que un transpilador sea considerado una utilidad productiva de estándar industrial, los errores de validación arrojados por el compilador final de enlazado (rustc) no pueden permanecer vinculados a las capas crípticas de intermediación abstracta. El usuario debe experimentar los fallos sobre el archivo original.

1. **Exterminio del Espacio Geográfico Estático:** El agente localizará a través del archivo de transpilación `resp-transpiler` todas las instanciaciones parametrizadas que convoquen ciegamente al método posicional universal `proc_macro2::Span::call_site()`. El proceso sistémico de mapeo debe integrarse para vincular el nodo subyacente envuelto en la variante original `Located<T>`. Aunque la traducción de metadatos de cuasi-comillas en archivos transitorios carece de una resolución nativa perfecta para `pragma posicional`, el transpilador forzará la creación paralela de un registro de mapeo físico, escupiendo un

diccionario estructural en formato sourcemap.json hacia el sistema de archivos del anfitrión durante cada compilación válida que relacione coordinadas interrelacionadas.

2. **Intercepción y Traducción Computacional de Fallos CLI:** El ejecutable principal empaquetado en crates/resp-cli/src/main.rs deberá refinar su invocación primitiva al proceso subyacente del compilador oficial.
 - La ejecución comandada a través del proceso estándar incorporará obligatoriamente el argumento adicional y forzoso --error-format=json, forzando a rustc a estructurar algorítmicamente todas sus advertencias espaciales de diagnóstico y prevención.
 - En caso de interrupción adversa del proceso (!status.success()), el manejador de la interfaz de la consola capturará el flujo de salida de error estándar. Posteriormente, decodificará jerárquicamente las tramas JSON arrojadas, consultará y filtrará el archivo posicional paralelo sourcemap.json recién materializado, y sustituirá de manera fluida y cosmética el nombre del archivo intermediario fallido por la línea equivalente exacta registrada en el archivo inicial con extensión .resp, publicando subsecuentemente la advertencia limpia y trazable en la consola del operador de software.

Fase 7: Validación de Madurez Autónoma y Culminación

Como un pilar de la finalización integral para evitar la persistencia en estado Alfa, las reglas exigen verificación comprobable.

1. **Pruebas de Integración (Integration Test Suites):** El asistente formulará una batería exhaustiva de vectores de prueba en el directorio raíz tests/ que verifiquen funcionalmente e independientemente los aspectos de *Herencia de Estructuras mediante Etendu, Llamadas Dinámicas Sometidas a Mangling de Sobrecarga, y Supresión de Pánico Interceptada en Bloques Provu-Kaptu*.
2. **Ciclo de Cierre iterativo:** El agente disparará el subprocesso validador cargo test --workspace. Si la biblioteca de gramática PEG rechaza las variantes recién introducidas o los generadores quote! producen flujos de tokens oxidados no legibles, la interfaz inteligente utilizará la retroalimentación extraída del gancho condicional temporal (PostToolUse) para corregir sus propias discrepancias cíclicamente y perfeccionar las expresiones subyacentes, culminando exitosamente en un compilador fuente a fuente estabilizado, completamente ergonómico y alineado con los estrictos requerimientos de documentación requeridos para indexación sin errores en el ecosistema mundial operado por la red neuronal profunda de Cognition AI y la infraestructura global de DeepWiki.

Fuentes citadas

1. DeepWiki: AI-Driven Revolution in Code Documentation - DEV Community, <https://dev.to/czmilo/deepwiki-ai-driven-revolution-in-code-documentation-1jb4>
2. DeepWiki and the New Way to Understand — and Reuse — Code | by Dario Fabiani, <https://medium.com/@dario.fabiani/deepwiki-and-the-new-way-to-understand-and-reuse-code-f0fae8163f2b>
3. The DeepWiki MCP Server - Cognition, <https://cognition.ai/blog/deepwiki-mcp-server>
4. DeepWiki: Understand Any Codebase - Hacker News, <https://news.ycombinator.com/item?id=45002092>